

Lista 04 - Victor Yaegashi Setti - RA 206362

Questão 1

a) Relação:

Seja i o número de meses e $T(i)$ o custo ótimo para este i :

$$T(i) = \begin{cases} 0, & \text{se } i \leq 0 \\ \min(T(i-1) + r_n \cdot m_n, T(i-6) + 6 \cdot c), & \text{se } i > 0 \end{cases}$$

b) Algoritmo:

```
CustoMinimo(c, n, M, R):  
01  DP = empty_vector(n+1)  
02  
03  for (i = 1; i <= n; i++):  
04      if (i < 6):  
05          temp = 6 * c  
06      else:  
07          temp = DP[i - 6] + (6 * c)  
08      DP[i] = min(temp, DP[i - 1] + (R[i] * M[i]))  
09  return DP[n]
```

Questão 2

- Supondo que a função `sort(A, m)` ordena o vetor `A` de tamanho `m` em $O(m \cdot \log m)$ operações como, por exemplo, o Merge Sort

```
Elemento6(V, n):  
01  S = V[0:10]  
02  sort(S, 11)  
03  return S[5]
```

Questão 3

- Supondo que a função `sort(A, m)` ordena o vetor `A` de tamanho `m` em $O(m \cdot \log m)$ operações como, por exemplo, o Merge Sort

```
MinIntervalos(V, n):  
01  sort(V, n)  
02  s = 0  
03  max = -INFINITE  
04  for (i = 1; i <= n; i++):  
05      if V[i] > max:  
06          s++  
07          max = V[i] + 1  
08  return s
```

Prova:

Teoria

Seja x_1 o menor elemento do vetor V e o intervalo $Q_x = [x_1, x_1 + 1] \subset \mathbb{R}$:

Considerando $S_0 = \{Q_1, Q_2, \dots, Q_s\}$ uma das soluções ótimas do problema e que, assim, existe Q_j , onde $1 \leq j \leq s$, que cobre x_1 .

- Se $Q_j = Q_x$, então Q_x satisfaz a condição e a solução está correta.
- Se não:
Temos então que $Q_j = [y, y + 1]$, onde $y < x_1$.
Como Q_x cobre x_1 e $(y + 1 < x_1 + 1)$, então Q_x cobre o mesmo número de valores do vetor V quanto Q_j .
 $\Rightarrow$ Seja a solução $S = [(S_0 \cup \{Q_x\}) \setminus Q_j]$, teremos que $|S| = |S_0|$, o que garante que S seja também solução ótima para o problema.

Algoritmo

Partindo da validade da prova anterior:

Selecionando o intervalo Q_{x_1} , todos os elementos $v \in V \mid x_1 \leq v \leq x_1 + 1$ são cobertos.

Portanto, restam a serem cobertos os elementos do subconjunto $V' = [V \setminus \{v \in V \mid x_1 \leq v \leq x_1 + 1\}]$.

A estrutura para encontrar uma solução ótima para V' é exatamente igual à para V , portanto, a escolha gulosa de Q_{x_1} simplifica o problema para um subproblema menor que

pode ser resolvido de mesma maneira.

Seguindo essa lógica indutiva a cada iteração do laço, temos que a sequência de escolhas gulosas que serão feitas geram uma solução ótima para o conjunto completo V .

Questão 4

- Supondo que a função `OrdenaPorDensidade(n, g, s)` ordena em ordem **decrecente** o vetor de conjuntos de dados utilizando a densidade $\frac{s_i}{g_i}$ como critério e retorna um vetor `v` tal que `v[i].g` corresponde ao tamanho g_i
- Supondo que a função `OrdenaPorDensidade(n, g, s)` possui complexidade temporal $O(n \cdot \log n)$

```
EscolheDados(n, g, s, N):  
01  f = empty_vector(n+1)  
02  V = OrdenaPorDensidade(n, g, s)  
03  
04  for (i = 1; i <= n; i++):  
05      if (V[i].g <= N):  
06          f[i] = 1  
07          N = N - V[i].g  
08      else:  
09          f[i] = (N / V[i].g)  
10          N = 0  
11          break  
12  return f
```

Complexidade Temporal

Considerando uma operação como um acesso a um item de um vetor, teremos:

- Alocação do vetor vazio: $T_1 = O(n)$
- `OrdenaPorDensidade(n, g, s)`: $T_2 = O(n \cdot \log n)$
- Loop: $T_3 = O(n)$

$$\Rightarrow T_{total} = T_1 + T_2 + T_3 = O(n) + O(n \cdot \log n) + O(n) = O(n \cdot \log n)$$

Questão 5

- Supondo que a função `empty_matrix(i, j)` aloca uma matrix zerada de `i` linhas e `j` colunas

- Para fins de simplificação adotaremos $c[i] = c_i$ e $g[i] = g_i$

```

EscolheDadosPD(n, g, s, N, K):
01 PD = empty_matrix(n + 1, N^K)
02 for (i = 1; i <= n; i++):
03     for ([c1, c2, ..., cK] ∈ [0, 1, ..., N - 1]^K):
04         melhor = PD[i - 1][c1][c2][...][cK]
05         for (j = 1; j <= K; j++):
06             if (g_i <= c_j):
07                 melhor = max(melhor, PD[i - 1][c1][c2][...]
                                [c_j - g_i][...][cK] + s_i)
08         PD[i][c1][c2][...][cK] = melhor
09 return PD[n][N - 1][N - 1][...][N - 1]

```

Complexidade Temporal

Considerando uma operação um acesso a um item de uma matriz, teremos:

- Alocação da matriz vazia: $T_1 = O((n + 1)N^K)$
- Loop interno: $T_2 = K \times O(1)$
- Loop intermediário: $T_3 = O(N^K) \times T_2 = O(KN^K)$
- Loop externo: $T_4 = O(n) \times T_3 = O(n) \times O(KN^K) = O(nKN^K)$

$$\Rightarrow T_{total} = T_1 + T_4 = O((n + 1)N^K) + O(nKN^K) = O(nKN^K)$$

Questão 6

- Sendo A uma árvore de n nós
- Supondo que a função $PegaFolhas(A, n)$ retorna um vetor v com todas as folhas da árvore A
- Supondo que a função $RemoveFolhas(A, n)$ remove da árvore A todas as folhas e seus respectivos pais

```

ConjuntoIndepMax(A, n):
01 S = empty_vector(n)
02 while (A != NULL):
03     F = PegaFolhas(A, n)
04     S = S U F
05     RemoveFolhas(A, n)
06 return S

```

Estrutura de Subproblemas Ótimos

Seja f um nó da árvore A e p o seu pai:

Ao inserir f na solução S , pela definição, p não poderá pertencer a S , o que resolve o subproblema ótimo para o que restou da árvore $A' = (A \setminus \{v, p\})$.

⇒ Combinando a escolha ótima local de incluir a folha f com a solução ótima do subproblema, temos assegurada a otimalidade global.

Prova da Estratégia Gulosa

Seja S_0 solução ótima, f um nó da árvore A e p o seu pai:

- Se $f \in S_0$ e $p \notin S_0$, então a escolha de f está correta e é ótima
- Se $f \notin S_0$ e $p \in S_0$:
Podemos definir $S = [(S_0 \setminus \{p\}) \cup \{f\}]$ pois, ao remover p e adicionar f à solução a independência dos nós é mantida.
Como $|S| = |S_0|$, temos que a escolha gulosa do algoritmo sempre resulta em folhas pertencentes à soluções ótimas e, portanto, chega em soluções ótimas.